



FACULDADE ESTÁCIO DE SÁ
CURSO: DESENVOLVIMENTO FULL STACK
5º SEMESTRE – MATRÍCULA 202302595341
Repositório GitHub - [alaimalmeida/softwareComSeguranca](https://github.com/alaimalmeida/softwareComSeguranca)
ALAIM ALMEIDA DE OLIVEIRA

Software sem segurança não serve!

Salvador – BA

2025

Introdução:

O projeto consiste em implementar um sistema de autenticação seguro utilizando uma interface de frontend simples e um backend que lida com a autenticação via JWT (JSON Web Token). O sistema permite o login com nome de usuário, e-mail e senha, além de incluir funcionalidades adicionais como visualização de perfil do usuário e lista de usuários para administradores.

Tecnologias Utilizadas

Backend:

- Node.js
- Express
- JWT (JSON Web Token)
- CORS
- Body-Parser

Frontend:

- HTML
- JavaScript
- CSS

Funcionalidades Implementadas

- Login Seguro

Permite login com nome de usuário ou e-mail e senha.

- Proteção com JWT

Autenticação nas rotas protegidas usando tokens JWT.

- Ações de Usuário

Botões 'Meu Perfil' e 'Ver Usuários' utilizam o token JWT para acessar rotas seguras.

- Implementação do Backend

Rota de login, autenticação com middleware, retorno de perfil e listagem de usuários (para administradores).

4. Estrutura de Dados

Usuário:

- id
- username
- email
- password
- role

Token JWT:

Contém id, username, email e role, com expiração de 1h.

5. Fluxo de Usuário

- Usuário preenche login.
- Frontend envia credenciais ao servidor.
- Servidor retorna token JWT.
- Token é armazenado no localStorage.
- Usuário pode acessar recursos protegidos.

6. Melhorias Futuras

- Criptografia de senha
- Banco de dados real
- Validação de entrada
- Expiração e revogação de token

7. Importância de um Projeto com Autenticação Segura

A segurança da autenticação é um dos pilares fundamentais de qualquer aplicação web. Sistemas que lidam com informações sensíveis, mesmo em pequena escala, precisam garantir que apenas usuários autorizados tenham acesso aos dados e funcionalidades disponíveis.

Neste projeto, a autenticação é realizada por meio de tokens JWT, o que proporciona diversas vantagens:

- O token é gerado no backend e contém informações criptografadas sobre o usuário.
- Cada requisição a rotas protegidas exige a presença de um token válido, aumentando a segurança.
- A autenticação via JWT reduz o risco de sequestro de sessão (session hijacking) comum em sistemas baseados apenas em cookies.
- O backend pode verificar facilmente se o token é válido e se o usuário tem permissão para acessar os recursos solicitados.

Ter um sistema com esse nível de segurança é essencial para proteger os dados dos usuários, manter a integridade do sistema e estar em conformidade com boas práticas e normas de proteção de dados, como a LGPD (Lei Geral de Proteção de Dados).

Contextualização do projeto

O time de segurança da Software House, onde você atua como Especialista em Desenvolvimento de Software, identificou uma falha de segurança, explorada por ataques que geraram o vazamento de dados, além de outros problemas, em uma das aplicações legadas, desenvolvida há alguns anos atrás. Tal falha consiste na concessão de acesso não autorizado de recursos a usuários.

Segue abaixo os prints do código completo - Backend:

Página index.html

```
index.html X
softwareComSeguranca > public > index.html > html > body > div.container
1  <!DOCTYPE html>
2  <html lang="pt-BR">
3  <head>
4    <meta charset="UTF-8" />
5    <title>Login Seguro</title>
6    <link rel="stylesheet" href="../components/style.css">
7
8  </head>
9  <body>
10   <div class="container">
11     <h1>Login</h1>
12
13     <form id="login-form">
14       <input type="text" id="username" placeholder="Usuário ou E-mail" required>
15       <input type="password" id="password" placeholder="Senha" required>
16       <button type="submit">Entrar</button>
17     </form>
18
19     <h2>Ações</h2>
20     <button id="btn-profile">Meu Perfil</button>
21     <button id="btn-users">Ver Usuários</button>
22
23     <div id="result"></div>
24   </div>
25
26   <script src="../components/script.js"></script>
27 </body>
28 </html>
29
```

Componente script

```
JS script.js X
softwareComSeguranca > components > JS script.js > addEventListener('submit') callback > catch() callback
1  document.getElementById('login-form').addEventListener('submit', function(event) {
2    event.preventDefault();
3
4    const username = document.getElementById('username').value;
5    const password = document.getElementById('password').value;
6
7    fetch('http://localhost:3000/api/auth/login', {
8      method: 'POST',
9      headers: {
10        'Content-Type': 'application/json',
11      },
12      body: JSON.stringify({ username, email: username, password }) // Enviando username ou email
13    })
14    .then(response => response.json())
15    .then(data => {
16      if (data.token) {
17        // Armazenar o token no localStorage ou sessionStorage
18        localStorage.setItem('token', data.token);
19        document.getElementById('result').textContent = 'Login realizado com sucesso!';
20      } else {
21        document.getElementById('result').textContent = 'Credenciais inválidas';
22      }
23    })
24    .catch(error => {
25      document.getElementById('result').textContent = 'Erro ao conectar com o servidor';
26    });
27 });
28
```

```
JS script.js X
softwareComSeguranca > components > JS script.js > addEventListener('submit') callback > catch() callback

29 // Ação para visualizar o perfil
30 document.getElementById('btn-profile').addEventListener('click', function() {
31   const token = localStorage.getItem('token');
32   if (token) {
33     fetch('http://localhost:3000/api/users/me', {
34       method: 'GET',
35       headers: {
36         'Authorization': `Bearer ${token}`
37       }
38     })
39     .then(response => response.json())
40     .then(data => {
41       document.getElementById('result').textContent = `Perfil: ${data.username} - ${data.email}`;
42     })
43     .catch(error => {
44       document.getElementById('result').textContent = 'Erro ao carregar perfil';
45     });
46   } else {
47     document.getElementById('result').textContent = 'Você precisa estar logado';
48   }
49 });
50
```

```
JS script.js X
softwareComSeguranca > components > JS script.js > addEventListener('submit') callback > catch() callback

51 // Ação para ver os usuários
52 document.getElementById('btn-users').addEventListener('click', function() {
53   const token = localStorage.getItem('token');
54   if (token) {
55     fetch('http://localhost:3000/api/users', {
56       method: 'GET',
57       headers: {
58         'Authorization': `Bearer ${token}`
59       }
60     })
61     .then(response => response.json())
62     .then(data => {
63       const userList = data.map(user => `<p>${user.username} - ${user.email}</p>`).join('');
64       document.getElementById('result').innerHTML = userList;
65     })
66     .catch(error => {
67       document.getElementById('result').textContent = 'Erro ao carregar usuários';
68     });
69   } else {
70     document.getElementById('result').textContent = 'Você precisa estar logado';
71   }
72 });
```

Componente style

```
# style.css X
softwareComSeguranca > components > # style.css > .container

1 body {
2   font-family: 'Segoe UI', sans-serif;
3   background: #f4f6f8;
4   display: flex;
5   justify-content: center;
6   align-items: center;
7   height: 100vh;
8   margin: 0;
9 }
10
11 .container {
12   background: #fff;
13   padding: 2rem;
14   border-radius: 10px;
15   box-shadow: 0 5px 20px rgba(0,0,0,0.1);
16   max-width: 400px;
17   width: 100%;
18 }
19
20 h2 {
21   text-align: center;
22   margin-bottom: 1.5rem;
23 }
24
25 input {
26   width: 100%;
27   padding: 0.8rem;
28   margin: 0.5rem 0;
29   border-radius: 8px;
30   border: 1px solid #ccc;
31   font-size: 16px;
32 }
```

```
# style.css X
softwareComSeguranca > components > # style.css > .container

33 button {
34   width: 100%;
35   padding: 0.9rem;
36   background-color: #007bff;
37   color: white;
38   font-size: 16px;
39   border: none;
40   border-radius: 8px;
41   cursor: pointer;
42   margin-top: 1rem;
43 }
44
45 button:hover {
46   background-color: #0056b3;
47 }
48
49 .users {
50   margin-top: 1rem;
51   display: grid;
52   gap: 1rem;
53 }
54
55 .user-card {
56   background: #e9f0f8;
57   padding: 1rem;
58   border-radius: 8px;
59   box-shadow: 0 2px 8px rgba(0,0,0,0.1);
60   font-size: 15px;
61 }
62
63 .user-card h3 {
64   margin: 0 0 0.5rem;
65 }
66
67 .user-card p {
68   margin: 0.2rem 0;
69 }
70
```

```
# style.css X
softwareComSeguranca > components > # style.css > .status

71
72 .status {
73   margin-top: 1rem;
74   text-align: center;
75   color: red;
76   font-weight: bold;
77 }
```

Página server.js

```
JS server.js X
softwareComSeguranca > JS server.js > ...
1  const express = require('express');
2  const cors = require('cors');
3  const jwt = require('jsonwebtoken');
4  const bodyParser = require('body-parser');
5  const path = require('path');
6
7  const app = express();
8  const port = 3000;
9
10 app.use(cors());
11 app.use(express.json());
12 app.use(express.static(path.join(__dirname, 'public')));
13
14 const SECRET_KEY = 'chave_secreta';
15
16 // Banco de dados simulado
17 const users = [
18   { id: 1, username: 'admin', email: 'admin@example.com', password: 'admin123', role: 'admin' },
19   { id: 2, username: 'user', email: 'user@example.com', password: 'user123', role: 'user' },
20 ];
21
22 Windsurf: Refactor | Explain | Generate JSDoc | X
23 function generateToken(user) { Ctrl+I Windsurf Command
24   return jwt.sign({ id: user.id, username: user.username, email: user.email, role: user.role }, SECRET_KEY, { expiresIn: '1h' });
25 }
```

```
JS server.js X
softwareComSeguranca > JS server.js > authenticateToken > jwt.verify() callback
Windsurf: Refactor | Explain | Generate JSDoc | X
26 function authenticateToken(req, res, next) {
27   const authHeader = req.headers['authorization'];
28   const token = authHeader && authHeader.split(' ')[1];
29
30   if (!token) return res.status(401).json({ message: 'Token não fornecido' });
31
32   jwt.verify(token, SECRET_KEY, (err, user) => {
33     if (err) return res.status(403).json({ message: 'Token inválido' });
34     req.user = user;
35     next();
36   });
37 }
38
39 Windsurf: Refactor | Explain | Generate JSDoc | X
40 function authorizeAdmin(req, res, next) {
41   if (req.user.role !== 'admin') {
42     return res.status(403).json({ message: 'Acesso negado: apenas administradores' });
43   }
44   next();
45 }
46
47 // Endpoint de login (aceita username ou email)
48 app.post('/api/auth/login', (req, res) => {
49   const { username, email, password } = req.body;
50   const user = users.find(u =>
51     ((u.username === username || u.email === email) && u.password === password)
52   );
53   if (!user) {
54     return res.status(401).json({ message: 'Credenciais inválidas' });
55   }
56
57   const token = generateToken(user);
58   res.json({ token });
59 });
```

```
JS server.js X
softwareComSeguranca > JS server.js > ...
60
61 // Endpoint que retorna o próprio perfil do usuário logado
62 app.get('/api/users/me', authenticateToken, (req, res) => {
63   const user = users.find(u => u.id === req.user.id);
64   if (!user) {
65     return res.status(404).json({ message: 'Usuário não encontrado' });
66   }
67   res.json({ id: user.id, username: user.username, email: user.email, role: user.role });
68 });
69
70 // Endpoint que lista todos os usuários (apenas para admins)
71 app.get('/api/users', authenticateToken, authorizeAdmin, (req, res) => {
72   res.json(users.map(u => ({ id: u.id, username: u.username, email: u.email, role: u.role })));
73 });
74
75 // Página principal
76 app.get('/', (req, res) => {
77   res.sendFile(path.join(__dirname, 'public', 'index.html'));
78 });
79
80 app.listen(port, () => {
81   console.log(`Servidor rodando em http://localhost:${port}`);
82 });
83
```

Página de login – Frontend

The image displays four mockups of a login page, arranged in a 2x2 grid, showing different states of the application:

- Top Left Mockup:** Shows the initial login form with the title "Login". It has two input fields: one for "admin" and one for a password (masked with "*****"). Below the inputs is a blue "Entrar" button. Underneath is a section titled "Ações" with two blue buttons: "Meu Perfil" and "Ver Usuários". At the bottom, it says "Login realizado com sucesso!".
- Top Right Mockup:** Similar to the top left, but the input fields contain "admin@example.com" and "*****". The "Entrar" button is still blue.
- Bottom Left Mockup:** Shows the profile view. The input fields contain "admin@example.com" and "*****". The "Entrar" button is blue. The "Ações" section has two blue buttons: "Meu Perfil" and "Ver Usuários". At the bottom, it says "Perfil: admin - admin@example.com".
- Bottom Right Mockup:** Shows the user list view. The input fields contain "admin@example.com" and "*****". The "Entrar" button is blue. The "Ações" section has two blue buttons: "Meu Perfil" and "Ver Usuários". At the bottom, it lists "admin - admin@example.com" and "user - user@example.com".

Login

admin@example.com

Entrar

Ações

Meu Perfil

Ver Usuários

Credenciais inválidas